Herald

Herald — HelixQA Integration (Operator Guide)

Field	Value
Revision	1
Created	2026-05-30
Last modified	2026-05-30
Status	active
Status summary	Operator-facing guide for running autonomous anti-bluff QA against Herald with the HelixQA framework. Documents what HelixQA is, the `~/Projects` sibling layout (CONST-051 flat-layout, no nested submodule chains), the LLM/Vision bridge (`claude` on PATH wrapped as a `BridgedCLIProvider`), the two Herald test banks under `challenges/helixqa-banks/` (the pherald `/v1/*` API bank + the eight-flavor CLI bank), the `scripts/helixqa_run.sh` launcher (build → boot real PG + pherald serve → run banks → preserve evidence), the bank YAML structure, the evidence layout under `qa-results/helixqa/<run-id>/`, and where HelixQA fits as Herald's autonomous-QA single-source-of-truth.
Issues	none
Issues summary	—
Fixed	(n/a — new guide)
Continuation	bump when the api bank gains coverage of new pherald `/v1/*` routes (e.g. a future `/v1/subscribers` surface); bump when additional flavors ship serving subcommands worth an HTTPS probe; bump when `HELIXQA_AUTONOMOUS=1` becomes the default after the autonomous session is validated against a live `claude` + PG environment; bump when the release gate (`scripts/release.sh`) wires `helixqa_run.sh` in as a tag-time blocker.

Table of contents

What HelixQA is

HelixQA is an **anti-bluff QA orchestration framework** for cross-platform testing with real-time crash detection, step validation, evidence collection, and automated ticket generation. Its design centre is the Helix §11.4 Operative Rule — the same covenant Herald restates at §107: **the bar for shipping is not "tests pass" but "the end user can use the feature."** Every PASS HelixQA emits must carry positive runtime evidence captured during execution; a green summary line without that evidence is a critical defect.

HelixQA consumes **YAML test banks** that describe platform-targeted test cases (structure, not prose) and — in autonomous mode — drives an LLM-powered session that navigates the system under test, verifies documented features, hunts for bugs, and writes a QA report with evidence.

For Herald, HelixQA validates two surfaces:

1. **The pherald HTTP API** (`platforms: [api]`) — the live Gin `/v1/*` plane: the JWT-gated `POST /v1/events` ingestion route plus the `healthz` / `readyz` / `metrics` built-ins.
2. **The eight flavor binaries** (`platforms: [desktop]`) — `pherald`, `sherald`, `cherald`, `bherald`, `rherald`, `iherald`, `scherald`, `qaherald` — each exercised end-to-end as a compiled binary. This is the §107 "the binary actually runs for the end user" check that Herald has repeatedly regressed on (unit tests call the cobra constructors and bypass `main()`, so eager-dependency and flag-ordering bugs hide there).

Sibling layout (CONST-051)

HelixQA lives as a **sibling repository**, not a Herald submodule. Per CONST-051 (submodules-as-equal-codebase + flat-layout, no nested own-org submodule chains), the expected on-disk arrangement is:

```
~/Projects/
├── Herald/          ← this repo
│   └── challenges/helixqa-banks/   ← Herald's banks (loaded by helixqa)
├── helixqa/         ← the HelixQA framework
│   └── bin/helixqa  ← built by scripts/helixqa_run.sh if missing
├── Challenges/      ← digital.vasic.challenges (HelixQA dependency)
└── Containers/      ← digital.vasic.containers (HelixQA dependency)
```

The launcher resolves HelixQA at `~/Projects/helixqa` by default; override with the `HELIXQA_DIR` environment variable. HelixQA's autonomous session hard-codes its bank-discovery path to `<project>/challenges/helixqa-banks` — which is exactly where Herald's banks live.

The LLM / Vision bridge

HelixQA's autonomous session and its prose-step bank evaluation are LLM-driven. It discovers providers two ways:

- **API-key providers** — `ANTHROPIC_API_KEY`, `OPENAI_API_KEY`, `OPENROUTER_API_KEY`, `GROQ_API_KEY`, … (40+ supported via the registry in `pkg/llm`).
- **Bridged CLI providers** — when a CLI tool such as `claude`, `qwen-coder`, or `opencode` is on `PATH`, HelixQA wraps it in a `BridgedCLIProvider` (zero API-key cost). The `claude` bridge is also **vision-capable**, so it doubles as the screenshot/visual analyzer.

Anti-bluff hard gate. `scripts/helixqa_run.sh` **FAILS LOUD if** `claude` **is not on** `PATH`**.** Rationale: with no provider at all, the autonomous session degrades to an observation-only mode that can print a green line *without* doing any real anti-bluff verification — exactly the "absence-of-error PASS" the §107 covenant forbids. Refusing to start is the honest behaviour. Install Claude Code and confirm `command -v claude` resolves before running.

Running the QA

```
# From the Herald repo root:
scripts/helixqa_run.sh
```

What the launcher does, in order:

1. **Preflight** — fail loud if `claude` is not on `PATH`; verify both banks exist.
2. **Build helixqa** — if `~/Projects/helixqa/bin/helixqa` is missing, `go build ./cmd/helixqa` (build log captured).
3. **Build all 8 flavor binaries** into the run's `bin/` directory so the CLI bank exercises freshly-compiled binaries.
4. **Boot a real** `pherald serve` for the API bank — mirroring `scripts/e2e_bluff_hunt.sh` (E7–E12): when Postgres is reachable on `:24100`, start `pherald serve` with `HERALD_PG_DSN` + `HERALD_AUTH_MODE=hmac` + `HERALD_AUTH_HMAC_SECRET`, wait for the **HTTPS** listener (Wave 4a — probe with `curl -k`), and capture a live `healthz` body as evidence. When PG is unreachable, the live serve is **SKIPPED with a recorded reason** (§11.4.3) and the API bank is left out of the run so no API coverage is claimed that was not exercised.
5. **Run the banks** — `helixqa run --banks <api,cli> --platform all` writing report + evidence under `qa-results/helixqa/<run-id>/`.
6. **(Optional) autonomous session** — set `HELIXQA_AUTONOMOUS=1` to also run `helixqa autonomous --project <repo> --platforms api,desktop`.
7. **Graceful teardown** — the `pherald serve` process is SIGTERM'd on exit. Any FAIL makes the script exit non-zero (release-gate composable).

Environment overrides: `HELIXQA_DIR`, `HERALD_HTTP_PORT` (default 24791), `HERALD_PG_DSN`, `HERALD_PG_PORT` (default 24100), `HERALD_AUTH_MODE`, `HERALD_AUTH_HMAC_SECRET`.

The test banks

Banks live under `challenges/helixqa-banks/` and follow the HelixQA YAML schema: top-level `version` / `name` / `test_cases[]`; each case carries `id` / `name` / `category` / `priority` / `platforms[]` / `steps[]` (each step `name` / `action` / `expected`) / `tags[]` / `documentation_refs[]`.

Bank	Platforms	Cases	Coverage
`herald-api-v1.yaml`	`api`	11	Real pherald `/v1/*` routes: `healthz`, `readyz`, `metrics`, and `POST /v1/events` (no-token 401, forged-token 401, valid-token 202 + Receipt, idempotency replay 200 + `X-Herald-Replay`, malformed body 400 `event_parser:`, missing-tenant-claim 401, 404 boundary, `/v1/compliance` not-served-by-pherald).
`herald-cli-flavors.yaml`	`desktop`	10	All 8 flavor binaries' `version --json` canonical shape + human DisplayName line; `pherald --help` subcommand discoverability; unknown-subcommand fail-loud.

Anti-bluff anchor. Every case asserts on the actual response **body** or **stdout** — `status:ok` + `build.version`, `status:ready`, the `pherald_build_info{` gauge line, the `event_parser:`-tagged error string, the `X-Herald-Replay` header, `binary == "<flavor>"` in `version --json`, the DisplayName on the first stdout line — never a bare 2xx-or-no-error check. The bank `expected` prose drives the LLM-generated verification prompts at runtime (CONST-046): banks describe **what correct looks like**, not a hard-coded English string match.

Evidence layout

Each run writes a timestamped directory:

```
qa-results/helixqa/<run-id>/
├── bin/                    freshly-built flavor binaries used by the CLI b
├── helixqa_build.log       helixqa go-build log (only if it had to be buil
├── flavor_build.log        go-build log for the 8 flavors
├── pherald_serve.log       stdout/stderr of the live pherald serve (api ba
├── healthz_evidence.json   captured live /v1/healthz body (positive eviden
├── api_serve_skip.txt      present only when PG was unreachable (SKIP reas
├── helixqa_run.log         the helixqa run transcript
├── helixqa_autonomous.log  present only with HELIXQA_AUTONOMOUS=1
└── qa-report.{md,html,json} + tickets/   HelixQA report + any generated t
```

This satisfies Herald §107.x (the `docs/qa/` / `qa-results/` evidence mandate): the run-id directory is the auditable runtime proof that the banks were exercised against real services and real binaries.

Where HelixQA fits — the autonomous-QA SSoT

HelixQA is Herald's **autonomous-QA single-source-of-truth**. It is complementary to, not a replacement for, the existing seams:

- `scripts/e2e_bluff_hunt.sh` — the canonical hand-rolled end-to-end smoke (14+ invariants against real services). HelixQA's launcher deliberately mirrors its PG + Gin boot so the two agree on environment.
- `tests/test_*_mutation_meta.sh` — the paired §1.1 mutation gates.
- **HelixQA** — the LLM-driven autonomous layer: it can grow coverage by exploration (curiosity phase), generate tickets for failures, and is the intended home for cross-flavor / cross-platform QA as Herald adds serving flavors and messenger channels (Wave 7+).

The intended end state is for `helixqa_run.sh` to be a release-gate blocker alongside the e2e smoke — a tag cannot ship if the banks FAIL.

Troubleshooting

Symptom	Cause / fix
`FAIL: \claude` is not on PATH`	Install Claude Code; confirm `command -v claude`. This is intentional — see the bridge section.
`HelixQA sibling repo not found at …`	Clone HelixQA to `~/Projects/helixqa` (CONST-051 flat layout) or set `HELIXQA_DIR`.
`SKIP api-serve: Postgres :24100 unreachable`	Start the quickstart Postgres (`podman-compose -f quickstart/docker-compose.quickstart.yml up -d postgres`) or point `HERALD_PG_DSN` / `HERALD_PG_PORT` at a reachable instance. The CLI bank still runs.
`pherald serve never accepted HTTPS within 10s`	Inspect `qa-results/helixqa/<run-id>/pherald_serve.log` — usually a JWT-verifier or DSN misconfig. The listener is HTTPS; probe with `curl -k`.
`go build failed for flavor …`	Inspect `flavor_build.log`. A fresh clone needs `go work init && go work use ./...` (go.work is gitignored per spec §9.1).

Sources verified

Per Helix §11.4.99, the external-tool behaviour documented here was cross-referenced against the in-tree HelixQA sources at the time of writing (no external network docs were required — HelixQA is a sibling repo on disk):

- `~/Projects/helixqa/README.md` — bank format, autonomous session, `BridgedCLIProvider`, CLI subcommands. Verified 2026-05-30.
- `~/Projects/helixqa/cmd/helixqa/main.go` — `run` / `autonomous` flags, the `<project>/challenges/helixqa-banks` discovery path, the claude/qwen-coder/opencode bridge discovery loop. Verified 2026-05-30.
- `~/Projects/helixqa/pkg/config/config.go` — the `api` / `desktop` platform constants. Verified 2026-05-30.